LAB MANUAL

ON

INFORMATION RETRIEVAL LAB MANUAL

ACADEMIC YEAR 2026-27

III B. Tech–I SEMESTER (R23)



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

V S M COLLEGE OF ENGINEERINGAUTONOMOUS
RAMCHANDRAPURAM
E.G DISTRICT
533255

B. Tech CSE (AI&ML) (R23) COURSE STRUCTURE & SYLLABUS
(Applicable from the academic year 2023-24 and onwards)

III B. Tech I Semester	INFORMATION RETRIEVAL LAB	L	T	P	C
		0	0	3	1.5

Course Outcomes: On completion of this course, the student will be able to

- Compute the similarity bet ween text documents
- Apply all pre-processing steps for text-data
- Implement classification of text documents.
- Perform document clustering using different algorithms.
- Implement PageRank algorithm for any network.

Programming Language: Python/R

Lab Experiments:

1. Representation of a Text Document in Vector Space Model and Computing Similarity between two documents.
2. Pre-processing of a Text Document: stop word removal and stemming
3. Construction of an Inverted Index for a given document collection comprising of at least 50 documents with a total vocabulary size of at least 1000 words.
4. Classification of a set of Text Documents into known classes (You may use any of the Classification algorithms like Naive Bayes, Max Entropy, Rochio's, Support Vector Machine). Standard Datasets will have to be used to show the results.
5. Text Document Clustering using K-means. Demonstrate with a standard dataset and compute performance measures- Purity, Precision, Recall and F-measure.
6. Crawling/ Searching the Web to collect news stories on a specific topic (based on user input). The program should have an option to limit the crawling to certain selected websites only.
7. To parse XML text, generate Web graph and compute topic specific page rank
8. Implement Matrix Decomposition and LSI for a standard dataset.
9. Mining Twitter to identify tweets for a specific period (and/or from a geographical location) and identify trends and named entities.
10. Implementation of PageRank on Scholarly Citation Network.

Experiment-1

1. **Representation of a Text Document in Vector Space Model and Computing Similarity between two documents.**

Aim: To Representation of a Text Document in Vector Space Model and Computing Similarity between two documents

Procedures:

- Representing text documents using the Vector Space Model (VSM).
- Computing similarity between two documents using Cosine Similarity.

Vector Space Model:

A vector space model is an algebraic model, involving two steps, in first step we represent the text documents into vector of words and in second step we transform to numerical format so that we can apply any text mining techniques such as information retrieval, information extraction, information filtering etc.

Ex:

Document 1: Cat runs behind rat

Document 2: Dog runs behind cat

Query: rat

Setp-1:

Document 1: (cat, runs, behind, rat)

Document 2: (Dog, runs, behind, cat) Query: (rat)

Step-2:

Term Document Matrix:

A term document matrix is a way of representing documents vectors in a matrix format in which each row represents term vectors across all the documents and columns represent document vectors across all the terms. The cell values frequency counts of each term in corresponding document. If a term is present in a document, then the corresponding cell value contains 1 else if the term is not present in the document, then the cell value contains 0.

After creating the term document matrix, we will calculate term weights for all the terms in the matrix across all the documents. It is also important to calculate the term weightings because we need to find out terms which uniquely define a document.

Tf-idf = tf X idf

tf = term frequency is the number of times a term occurs in a document

idf = inverse of the document frequency, given as below

idf = log(N/df), where df is the document frequency-number of documents containing a term

total documents (N)
2

Term document matrix			
words\documents	Document1	document2	query term
cat	1	1	0
runs	1	1	0
behind	1	1	0
rat	1	0	1
dog	0	1	0

idf calculation	
document frequency (df)	idf - log(N/df)
2	0
2	0
2	0
1	0.30103
1	0.30103

Term document matrix with tf-idf			
words\documents	Document1	document2	query term
cat	0	0	0
runs	0	0	0
behind	0	0	0
rat	0.30103	0	0.30103
dog	0	0.30103	0

Note, there are many variations in the way we calculate the term-frequency(tf) and inverse document frequency (idf), in this post we have seen one variation. Below images show as the other recommended variations of tf and idf,

Variants of term frequency (TF) weight

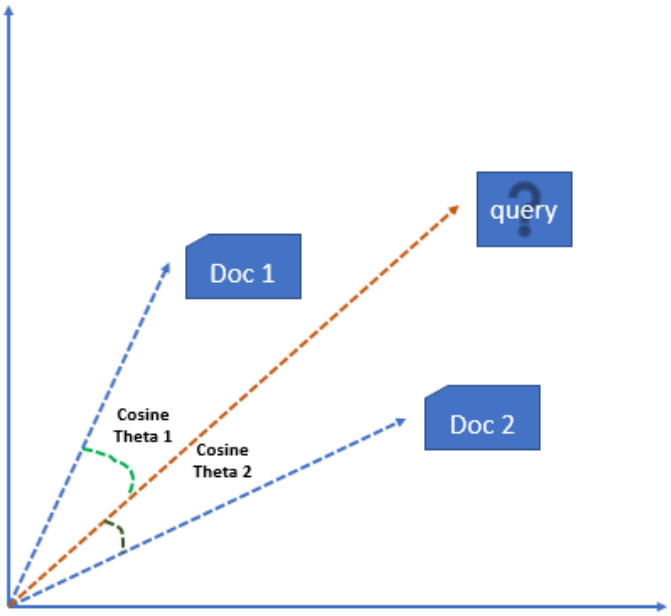
weighting scheme	TF weight
binary	$0, 1$
raw count	$f_{t,d}$
term frequency	$f_{t,d} / \sum_{t' \in d} f_{t',d}$
log normalization	$1 + \log(f_{t,d})$
double normalization 0.5	$0.5 + 0.5 \cdot \frac{f_{t,d}}{\max_{\{t' \in d\}} f_{t',d}}$
double normalization K	$K + (1 - K)\frac{f_{t,d}}{\max_{\{t' \in d\}} f_{t',d}}$

Variants of inverse document frequency (IDF) weight

weighting scheme	IDF weight ($n_t = \|\{d \in D : t \in d\}\|$)
unary	1
inverse document frequency	$\log \frac{N}{n_t} = -\log \frac{n_t}{N}$
inverse document frequency smooth	$\log\left(1 + \frac{N}{n_t}\right)$
inverse document frequency max	$\log\left(\frac{\max_{\{t' \in d\}} n_{t'}}{1 + n_t}\right)$
probabilistic inverse document frequency	$\log \frac{N - n_t}{n_t}$

Similarity Measures: cosine similarity:

Mathematically, closeness between two vectors is calculated by calculating the cosine angle between two vectors. In similar lines, we can calculate cosine angle between each document vector and the query vector to find its closeness.



Program:

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

def calculate_document_similarity(documents):
    vectorizer = TfidfVectorizer()
    tfidf_matrix = vectorizer.fit_transform(documents)
    similarity_matrix = cosine_similarity(tfidf_matrix)
    return similarity_matrix

if __name__ == "__main__": # Corrected this line
    doc1 = "cat runs behind rat"
    doc2 = "dog runs behind cat"
    query = "rat" # Removed extra space before 'query'

    corpus = [doc1, doc2, query]

    similarity_scores = calculate_document_similarity(corpus)

    print("Similarity Matrix:")
    print(similarity_scores)

    print(f"\nSimilarity between Document 1 and Document 2: {similarity_scores[0, 1]:.4f}")
    print(f"Similarity between Document 1 and query: {similarity_scores[0, 2]:.4f}")
    print(f"Similarity between Document 2 and query: {similarity_scores[1, 2]:.4f}")
```

Output:

Similarity Matrix:

```
[[1.         0.68978029 0.5       ]
 [0.68978029 1.         0.        ]
 [0.5        0.         1.        ]]
```

Similarity between Document 1 and Document 2: 0.6898

Similarity between Document 1 and query: 0.5000

Similarity between Document 2 and query: 0.0000

Experiment-2

2. Pre-processing of a Text Document: stop word removal and stemming

Aim: To processing of a Text Document: stop word removal and stemming

Procedure:

- **Text pre-processing** is the process of preparing text data so that machines can use the same to perform tasks like analysis, predictions, etc. There are many different steps in text pre-processing but in this article, we will only get familiar with stop words, why do we remove them, and the different libraries that can be used to remove them.
- **Stop words:** The words which are generally filtered out before processing a natural language are called **stop words**. These are actually the most common words in any language (like articles, prepositions, pronouns, conjunctions, etc) and does not add much information to the text. Examples of a few stop words in English are "the", "a", "an", "so", "what".
- Stop words are available in abundance in any human language. By removing these words, we remove the low-level information from our text in order to give more focus to the important information.
- **Removal of stop words** definitely reduces the dataset size and thus reduces the training time due to the fewer number of tokens involved in the training.
- **Stemming** is a technique that reduces a word to its root form. Although the stemmed word may not always be a real or grammatically correct word in English, it does help to consolidate different forms of the same word to a common base form, reducing the complexity of text data. For example, the words "run", "runs", "running" might all be stemmed to the common root "run". This helps our algorithm understand that these words are related and they carry a similar semantic meaning.

Program:

```
from sklearn.feature_extraction.text import ENGLISH_STOP_WORDS
from nltk.stem import PorterStemmer
import re
import nltk # Import NLTK
# Sample text document
text = "Running and jogging are great exercises. The runner was running swiftly." # 1. Preprocessing: lowercase and remove non-alphabetic characters
text = text.lower()
words = re.findall(r'\b[a-z]+\b', text)
# 2. Remove stopwords using sklearn's ENGLISH_STOP_WORDS
filtered_words = [word for word in words if word not in ENGLISH_STOP_WORDS] # 3. Stemming using NLTK's PorterStemmer
stemmer = PorterStemmer()
stemmed_words = [stemmer.stem(word) for word in filtered_words] # 4. Final result
print("Original words:", words)
print("Stopword removal:", filtered_words) # Minor capitalization for consistency
print("Stemmed:", stemmed_words)
```

Output:

Original words: ['running', 'and', 'jogging', 'are', 'great', 'exercises', 'the', 'runner', 'was', 'running', 'swiftly']

Stopword removal: ['running', 'jogging', 'great', 'exercises', 'runner', 'running', 'swiftly']

Stemmed: ['run', 'jog', 'great', 'exercis', 'runner', 'run', 'swiftli']

Experiment-3

3. Construction of an Inverted Index for a given document collection comprising of at least 50 documents with a total vocabulary size of at least 1000 words

Aim: To Construction of an Inverted Index for a given document collection comprising of at least 50 documents with a total vocabulary size of at least 1000 words

Procedure:

- An inverted index is an index data structure storing a mapping from content, such as words or numbers, to its locations in a document or a set of documents. In simple words, it is a hashmap like data structure that directs you from a word to a document or a web page.
- There are two types of inverted indexes:

  A **record-level inverted index** contains a list of references to documents for each word.

  A **word-level inverted index** additionally contains the positions of each word within a document.

  The latter form offers more functionality, but needs more processing power and space to be created.

Suppose we want to search the texts "hello everyone, " "this article is based on inverted index, " "which is hashmap like data structure". If we index by (text, word within the text), the index with location in text is:

Hello	(1,	1)
Everyone	(1,	2)
This	(2,	1)
Article	(2,	2)
Is	(2,	3); (3, 2)
Based	(2,	4)
On	(2,	5)
Inverted	(2,	6)
Index	(2,	7)
Which	(3,	1)
Hashmap	(3,	3)
Like	(3,	4)
Data	(3,	5)
Structure	(3,	6)

The word "hello" is in document 1 ("hello everyone") starting at word 1, so has an entry (1, 1) and word "is" is in document 2 and 3 at '3rd' and '2nd' positions respectively (here position is based on word).

The index may have weights, frequencies, or other indicators.

Steps to build an inverted index:

1. **Preprocess Documents**:
    - Lowercasing
    - Tokenization
    - Stopword removal
    - (Optional) Stemming or Lemmatization
2. **Create the Inverted Index**:
    - For each token in each document, add the document ID to the index under the token.
3. **Store Document IDs for each term**.

Program:

```
import re
import collections
from pprint import pprint


# --- 1. Document Collection ---
# A collection of 50 sample text documents.
# In a real-world scenario, these would be read from files.
document_collection = {
1: """The sun is a star, the center of our solar system.""",
    2: """The moon is Earth's only natural satellite.""",
    3: """Mars, often called the 'Red Planet', is the fourth planet from the Sun.""",
    4: """Jupiter is the largest planet in our solar system.""",
    5: """Saturn is the sixth planet from the Sun, famous for its extensive ring system.""",
    6: """A computer is a machine that can be programmed to carry out sequences of arithmetic or logical operations automatically.""",
    7: """Python is a high-level, interpreted programming language.""",
    8: """An algorithm is a set of rules or instructions specified in a sequence to solve a particular problem.""",
    9: """Data structures are a way of organizing and storing data in a computer so that it can be accessed and modified efficiently.""",
    10: """Machine learning is a field of artificial intelligence that uses statistical techniques to give computer systems the ability.""",
    11: """The history of the world is the story of human past. It is a field of study.""",
    12: """Ancient Greece was a civilization belonging to a period of Greek history.""",
    13: """The Roman Empire was the post-Republican period of ancient Rome.""",
    14: """The Renaissance was a period in European history marking the transition.""",
    15: """The Industrial Revolution was the transition to new manufacturing processes in Europe and the United States.""",
```

16: """Biology is the natural science that studies life and living organisms, including their physical structure, chemical processes.""",

17: """Genetics is a branch of biology concerned with the study of genes, genetic variation, and heredity in organisms.""",

18: """Ecology is the branch of biology which studies the interactions among organisms and their environment.""",

19: """Photosynthesis is a process used by plants and other organisms to convert light energy into.""",

20: """A cell is the basic structural and functional unit of all known organisms.""",

21: """Chemistry is the scientific discipline involved with elements and compounds composed of atoms, molecules and ions:""",

22: """An atom is the smallest unit of ordinary matter that forms a chemical element.""",

23: """A molecule is an electrically neutral group of two or more atoms held together by chemical bonds.""",

24: """The periodic table is a tabular arrangement of the chemical elements, ordered.""",

25: """A chemical reaction is a process that leads to the chemical transformation of one set of chemical substances to another.""",

26: """Physics is the natural science that studies matter, its motion and behavior through .""",

27: """Quantum mechanics is a fundamental theory in physics that provides a description.""",

28: """Relativity, as developed by Albert Einstein, encompasses two interrelated theories:.""",

29: """Newton's laws of motion are three physical laws that, together, laid the foundation for classical mechanics.""",

30: """Thermodynamics is a branch of physics that deals with heat, work, and temperature.""",

31: """Literature is any collection of written work, but it is also used more narrowly for writings specifically.""",

32: """William Shakespeare was an English playwright, poet, and actor, widely regarded as the greatest.""",

33: """A novel is a relatively long work of narrative fiction, typically written in prose and published as a book.""",

34: """Poetry is a form of literature that uses aesthetic and often rhy.""",

35: """Drama is the specific mode of fiction represented in performance: a play.""",

36: """Economics is the social science that studies the production, distribution, and consumption of goods and services.""",

37: """Microeconomics is a branch of economics that studies the behavior.""",

38: """Macroeconomics is a branch of economics dealing with the performance.""",

39: """A market is a composition of systems, institutions, procedures.""",

40: """Supply and demand is a model in microeconomics. It is a fundamental concept that describes how prices.""",

41: """Art is a diverse range of human activities in creating visual, auditory or performing artifacts (artworks).""",

42: """Painting is the practice of applying paint, pigment, color or other medium to a solid surface.""",

43: """Sculpture is the branch of the visual arts that operates in three dimensions. It is one of the plastic arts.""",

```
    44: """Music is the art of arranging sounds in time to produce a composition through.""",
    45: """Architecture is both the process and the product of planning, designing, and constructing buildings or other structures.""",
    46: """Philosophy is the study of general and fundamental questions.""",
    47: """Metaphysics is the branch of philosophy that examines the fundamental nature of reality.""",
    48: """Epistemology is the branch of philosophy concerned with knowledge.""",
    49: """Ethics or moral philosophy is a branch of philosophy that involves systematizing.""",
    50: """Logic is the systematic study of valid rules of inference, i.e. the relations that lead.""",
}
# --- 2. Pre-processing and Tokenization ---
def tokenize(document_text):
    """
    Converts a document string into a list of lowercase tokens (words),
    removing punctuation and simple stop words.
    """
    # Convert to lowercase and find all word sequences
    tokens = re.findall(r'\b\w+\b', document_text.lower())

    # A simple list of stop words to exclude.
    # A more comprehensive list could be used for better results.
    stop_words = set([
        'the', 'a', 'an', 'in', 'is', 'it', 'and', 'of', 'to', 'on', 'for',
        'with', 'as', 'by', 'that', 'this', 'was', 'were', 'be', 'are', 'i'
    ])

    # Filter out stop words
    return [token for token in tokens if token not in stop_words]


# --- 3. Inverted Index Construction ---
def build_inverted_index(docs):
    """
    Builds an inverted index from a collection of documents.

    The index is a dictionary where keys are terms (words) and values are
    'postings lists' - lists of document IDs where the term appears.
```

```
    """
    # Use a defaultdict to easily append to lists
    inverted_index = collections.defaultdict(list)

    # Process each document
    for doc_id, doc_text in docs.items():
        tokens = tokenize(doc_text)
        # For each unique token in the document, add the doc_id to its list
        for token in set(tokens): # Use set() to count each term only once per doc
            inverted_index[token].append(doc_id)

    # Sort the document IDs for each term for consistency
    for term in inverted_index:
        inverted_index[term].sort()

    return inverted_index


# --- Main Execution ---
if __name__ == "__main__":
    # Build the inverted index from our document collection
    inverted_index = build_inverted_index(document_collection)

    # --- 4. Displaying Results ---
    # Print the total number of documents and the vocabulary size
    all_tokens = [token for text in document_collection.values() for token in tokenize(text)]
    vocabulary = set(all_tokens)

    print(f"Number of documents: {len(document_collection)}")
    print(f"Vocabulary Size (number of unique words): {len(vocabulary)}\n")

    print("--- Inverted Index (Sample) ---")
    # Print a sample of the inverted index to show its structure
    sample_terms = list(inverted_index.keys())[50:60]
    for term in sample_terms:
        print(f"'{term}': {inverted_index[term]}")
```

```
print("\n--- Full Inverted Index ---")
# The full index can be very large, so we use pprint for better formatting.
# Uncomment the line below to print the entire inverted index.
# pprint.pprint(dict(inverted_index))

# Example: How to use the index
print("\n--- Example Usage ---")
search_term = 'science'
if search_term in inverted_index:
    doc_ids = inverted_index[search_term]
    print(f"The term '{search_term}' appears in documents: {doc_ids}")
else:
    print(f"The term '{search_term}' was not found in any document.")
```

Output:

```
Number of documents: 50
Vocabulary Size (number of unique words): 319

--- Inverted Index (Sample) ---
'problem': [8]
'algorithm': [8]
'specified': [8]
'instructions': [8]
'organizing': [9]
'efficiently': [9]
'way': [9]
'data': [9]
'modified': [9]
'storing': [9]

--- Full Inverted Index ---

--- Example Usage ---
The term 'science' appears in documents: [16, 26, 36]
```

Experiment-4

4. **Classification of a set of Text Documents into known classes (You may use any of the Classification algorithms like Naive Bayes, Max Entropy, Rochio's, Support Vector Machine). Standard Datasets will have to be used to show the results.**

Aim:

Program for Classification of a set of Text Documents into known classes (You may use any of the Classification algorithms like Naive Bayes, Max Entropy, Rochio's, Support Vector Machine). Standard Datasets will have to be used to show the results.

Procedure:

Text Classification Algorithms

- Naive Bayes:

  A probabilistic classifier based on Bayes' theorem, assuming feature independence. It's known for its simplicity and speed, making it a popular choice for text classification.

- Max Entropy:

  A flexible classifier that makes minimal assumptions about the data distribution. It can handle complex relationships between features and classes, making it suitable for text classification.

- Support Vector Machines (SVM):

  A powerful algorithm that finds the optimal hyperplane to separate data points into different classes. SVM is known for its ability to handle high-dimensional data and its effectiveness in various text classification tasks.

- Rocchio's Algorithm:

  A centroid-based algorithm that classifies documents based on their similarity to class centroids. It's relatively simple to implement and understand

Program:

```
from sklearn.datasets import fetch_20newsgroups
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, accuracy_score

# Load dataset (subset for simplicity)
categories = ['comp.graphics', 'sci.med', 'rec.sport.baseball', 'talk.politics.mideast']
newsgroups = fetch_20newsgroups(subset='all', categories=categories, remove=('headers', 'footers', 'quotes'))

# Preprocessing and Vectorization
vectorizer = TfidfVectorizer(stop_words='english', max_features=2000)
X = vectorizer.fit_transform(newsgroups.data)
y = newsgroups.target

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
```

```
# Choose a model
model = LogisticRegression(max_iter=1000)

# Train the model
model.fit(X_train, y_train)

# Predict and Evaluate
y_pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred,
target_names=newsgroups.target_names))
```

Output:

```
Accuracy: 0.8786324786324786

Classification Report:
                    precision    recall  f1-score   support

     comp.graphics       0.92      0.87      0.90       305
  rec.sport.baseball     0.81      0.93      0.86       304
           sci.med       0.87      0.89      0.88       290
talk.politics.mideast    0.95      0.82      0.88       271

          accuracy                           0.88      1170
         macro avg       0.89      0.88      0.88      1170
      weighted avg       0.88      0.88      0.88      1170
```

Experiment-5

5. **Text Document Clustering using K-means. Demonstrate with a standard dataset and compute performance measures- Purity, Precision, Recall and F-measure.**

Aim:

Text Document Clustering using K-means. Demonstrate with a standard dataset and compute performance measures- Purity, Precision, Recall and F-measure.

Procedure:

Clustering text documents is a common problem in Natural Language Processing (NLP) where similar documents are grouped based on their content. **K-Means clustering** is a popular clustering technique used for this purpose. In this article we'll learn how to perform text document clustering using the K-Means algorithm in Scikit-Learn.

Step-by-step Plan:

1. Load dataset (20 Newsgroups).
2. Convert text to numerical vectors using TF-IDF.
3. Apply **K-Means** clustering.
4. Evaluate performance using:
    - **Purity**
    - **Precision, Recall, F1-Score**

Program:

```
from sklearn.datasets import fetch_20newsgroups
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.cluster import KMeans
from sklearn.metrics import classification_report, confusion_matrix
import numpy as np
from scipy.optimize import linear_sum_assignment
from collections import Counter

# Step 1: Load the dataset
categories = ['alt.atheism', 'comp.graphics', 'rec.sport.baseball', 'sci.space']
data = fetch_20newsgroups(subset='all', categories=categories, remove=('headers', 'footers', 'quotes'))
true_labels = data.target
num_clusters = len(categories)

# Step 2: Convert text to TF-IDF vectors
vectorizer = TfidfVectorizer(stop_words='english', max_df=0.5)
X = vectorizer.fit_transform(data.data)
```

```
# Step 3: Apply KMeans
kmeans = KMeans(n_clusters=num_clusters, random_state=42, n_init='auto')
kmeans.fit(X)
predicted_labels = kmeans.labels_

# Step 4: Compute Purity Score
def purity_score(y_true, y_pred):
    """
    Calculates the purity score for a clustering result.
    """
    contingency_matrix = confusion_matrix(y_true, y_pred)
    return np.sum(np.amax(contingency_matrix, axis=0)) / np.sum(contingency_matrix)

purity = purity_score(true_labels, predicted_labels)
print(f"\nPurity Score: {purity:.4f}")

# Step 5: Compute Precision, Recall, F1 using Hungarian Matching
def best_map(y_true, y_pred):
    """
    Finds the best mapping between predicted and true labels using the Hungarian algorithm
    to correct for label permutation in unsupervised clustering.
    """
    cm = confusion_matrix(y_true, y_pred)
    row_ind, col_ind = linear_sum_assignment(-cm)
    mapping = dict(zip(col_ind, row_ind))
    new_preds = [mapping[label] for label in y_pred]
    return new_preds

mapped_preds = best_map(true_labels, predicted_labels)
print("\nClassification Report (after label mapping):")
print(classification_report(true_labels, mapped_preds, target_names=categories))
```

Output:

Purity Score: 0.7024

Classification Report (after label mapping):

	precision	recall	f1-score	support
alt.atheism	0.91	0.55	0.69	799
comp.graphics	0.51	0.95	0.66	973
rec.sport.baseball	1.00	0.61	0.76	994
sci.space	0.77	0.67	0.72	987
accuracy			0.70	3753
macro avg	0.80	0.70	0.71	3753
weighted avg	0.79	0.70	0.71	3753

Experiment-6

6. Crawling/ Searching the Web to collect news stories on a specific topic (based on user input). The program should have an option to limit the crawling to certain selected websites only

Aim:

Crawling/ Searching the Web to collect news stories on a specific topic (based on user input). The program should have an option to limit the crawling to certain selected websites only

Procedure:

A web crawler can be built using programming languages like Python and libraries like requests and Beautiful Soup or Scrapy to collect news stories on a specific topic from selected websites. The crawler can be configured to follow links on specified websites and extract relevant news articles, while also allowing the user to define a list of target websites to limit the search.

Here's a breakdown of the process:

1. Define Target Websites:
   - The user provides a list of URLs for the websites they want to crawl.
   - This list acts as a filter, ensuring the crawler only visits and extracts data from these specific sources.
2. Web Crawling with requests and Beautiful Soup:
   - requests library: Used to fetch the HTML content of web pages from the target websites.
   - Beautiful Soup library: Used to parse the HTML content and extract specific data elements, such as news article titles, summaries, and links.
   - The crawler follows hyperlinks on the initial pages and recursively explores linked pages within the defined websites.
   - A priority queue can be implemented to prioritize specific URLs (e.g., product pages).

Program:

```
import requests
from bs4 import BeautifulSoup
import re

def search_news(topic, sites=None, limit=10):
    # Construct the search query
  search_query = f"{topic}"
  if sites:
    # Add 'site:domain.com' for each specified site
    site_queries = " OR ".join([f"site:{site}" for site in sites])
    search_query += f" {site_queries}"
```

```
    print(f"Searching for: '{search_query}'...")

    # Use DuckDuckGo's HTML version which is simpler to scrape
    url = "https://html.duckduckgo.com/html/"
    headers = {
        'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/58.0.3029.110 Safari/537.36'
    }
    params = {'q': search_query}

    try:
        # Make the web request
        response = requests.post(url, headers=headers, data=params)
        response.raise_for_status()  # Raise an exception for bad status codes (4xx or 5xx)

        # Parse the HTML content
        soup = BeautifulSoup(response.text, 'html.parser')

        # Find all search result containers
        results = soup.find_all('div', class_='result')

        articles = []
        for result in results:
            if len(articles) >= limit:
                break

            # Extract title, link, and snippet from the result element
            title_tag = result.find('h2', class_='result__title')
            link_tag = result.find('a', class_='result__a')
            snippet_tag = result.find('a', class_='result__snippet')

            if title_tag and link_tag and snippet_tag:
                title = title_tag.get_text(strip=True)
                link = link_tag['href']
                snippet = snippet_tag.get_text(strip=True)

                # The links from DuckDuckGo are redirect URLs. We can try to clean them.
```

```
            # Example: /l/?kh=-1&uddg=https%3A%2F%2Fwww.example.com
            match = re.search(r"uddg=([^&]+)", link)
            if match:
                # URL decode the link
                from urllib.parse import unquote
                actual_link = unquote(match.group(1))
            else:
                actual_link = link

            articles.append({
                'title': title,
                'link': actual_link,
                'snippet': snippet
            })

        return articles

    except requests.exceptions.RequestException as e:
        print(f"An error occurred: {e}")
        return []

# --- Main Execution ---
if __name__ == "__main__":
    # Get user input for the topic
    topic_input = input("Enter the news topic you want to search for: ")

    # Get user input for specific websites (optional)
    sites_input = input("Enter websites to search within (comma-separated, e.g., bbc.com,reuters.com) or
press Enter to search all: ")

    # Convert the sites string into a list
    site_list = [site.strip() for site in sites_input.split(',')] if sites_input else None

    # Fetch the news articles
    found_articles = search_news(topic_input, sites=site_list)

    # Display the results
```

```
if found_articles:
        print(f"\n--- Found {len(found_articles)} articles ---")
        for i, article in enumerate(found_articles, 1):
            print(f"\n{i}. {article['title']}")
            print(f"   Link: {article['link']}")
            print(f"   Snippet: {article['snippet']}")
    else:
        print("\nNo articles found or an error occurred during search.")
```

Output:

Enter the news topic you want to search for: weather condition
Enter websites to search within (comma-separated, e.g., bbc.com,reuters.com) or press Enter to search all: bbc.com, cnn.com
Searching for: 'weather condition site:bbc.com OR site:cnn.com'...

--- Found 10 articles ---

1. Weather and forecasts | CNN
   Link: https://www.cnn.com/weather
   Snippet: Get the latestweathernews and forecasts from CNN's meteorologists, watch extremeweathervideos, learn about climate change and follow major hurricanes with CNN's storm tracker.

2. Breaking News, Latest News and Videos | CNN
   Link: https://www.cnn.com/
   Snippet: View the latest news and breaking news today for U.S., world,weather, entertainment, politics and health atCNN.com.

3. Henriette regains tropical storm strength in the Pacific but is still ...
   Link: https://www.cnn.com/2025/08/10/weather/henriette-tropical-storm-strength-hnk
   Snippet: Henriette regained tropical storm strength in the Pacific Ocean well east of Hawaii on Saturday, but forecasters said it was still not expected to pose a threat to land.

4. Milwaukee contends with flash flooding, state fair and airport ... - CNN
   Link: https://www.cnn.com/2025/08/10/weather/milwaukee-flooding-rain-airport
   Snippet: Excessive rainfall in Wisconsin caused flash flooding this weekend, leading to inundated roads,

flooded basements and an early closure of the region's beloved state fair.

5. A major cooldown is coming for the US, but flooding could return | CNN

Link: https://www.cnn.com/2025/07/29/weather/cooldown-pattern-change-heat-climate

Snippet: A majorweatherpattern change beginning later this week will bring high temperatures more like early June — nearly 20 degrees cooler than they are now — to millions currently reeling through ...

6. Tropical Storm Erin is expected to strengthen into the first hurricane ...

Link: https://edition.cnn.com/2025/08/11/weather/tropical-storm-erin-hurricane-season-climate

Snippet: Tropical Storm Erin is tracking west over the open Atlantic Ocean on Tuesday and forecasters expect it to become the first hurricane of the Atlantic season this week.

7. US heat risk forecast: See where Americans are at the most risk for ...

Link: https://edition.cnn.com/weather/extreme-heat-risk-tracker-dg

Snippet: CNN is tracking extreme heatconditionsfor Americans each day. See where the risks are greatest — and where high temperature records might be broken.

8. Why weather forecasters often get it wrong - or appear to - BBC

Link: https://www.bbc.com/news/articles/cwy1epz58pyo

Snippet: The accuracy of forecasting has improved enormously - and AI promises further advances. But there are certain challenges in finding ways to share that mass of data with viewers

9. Millions of Americans advised to stay indoors due to triple-digit ... - BBC

Link: https://www.bbc.com/news/articles/c80dl731pk1o

Snippet: Heat advisories affect 168 million Americans in the mid-west, south-east and parts of the north-east US.

10. Thanksgiving weekend weather: An Arctic blast looms for much of ... - CNN

Link: https://www.cnn.com/2024/11/28/weather/thanksgiving-weekend-weather-arctic-storm

Snippet: While the Northeast gets hammered by frigid rain or snow this Thanksgiving, much of the US can be thankful for calm, dryweather. But a bitter blast of Arctic air will envelop much of the country ...

Experiment-7

7. To parse XML text, generate Web graph and compute topic specific page rank

Aim: To parse XML text, generate Web graph and compute topic specific page rank

Procedure:

PageRank is an algorithm used by Google to rank web pages in search engine results. It assigns a numerical weight to each web page, indicating its importance based on the number and quality of backlinks. Pages with more high-quality backlinks from other important pages are ranked higher. The algorithm is iterative, meaning it refines its understanding of the web's structure over multiple calculations.

Here's a breakdown of how PageRank works:

1. Link Analysis: The core idea is that a page's importance is determined by the number and quality of other pages that link to it.
2. Backlinks and Quality: The algorithm considers both the number of backlinks a page receives and the PageRank of the pages providing those links.
3. Iterative Calculation: PageRank is not calculated in a single step. It starts with an initial estimate and iteratively refines the rank of each page until it converges to a stable state.
4. Damping Factor: To prevent issues like "spider traps" or "rank sinks" (where a group of pages might get stuck in an infinite loop), a damping factor is introduced. This factor acts like a probability that a random user will stop following links on a page and "jump" to a different page in the search space.
5. Importance of Quality Links: While the number of backlinks is a factor, the quality of those backlinks is crucial. A link from a highly reputable and authoritative website carries more weight than a link from a low-quality, spammy site.
6. Recursive Calculation: The PageRank of a page is calculated based on the PageRank of the pages linking to it, divided by the number of outbound links from those linking pages. The formula is often expressed as:

PR(A) = (1-d) + d * (PR(T1)/C(T1) + ... + PR(Tn)/C(Tn))

Where:

- PR(A) is the PageRank of page A.
- d is the damping factor (typically around 0.85).
- T1 to Tn are the pages linking to page A.
- C(T) is the number of outbound links on page T

Program:

```
import xml.etree.ElementTree as ET
import networkx as nx
from pprint import pprint

def parse_xml_to_graph(xml_string):

    # Create a new directed graph
    graph = nx.DiGraph()

    # Parse the XML string
    root = ET.fromstring(xml_string)

    # Iterate through each <page> element in the XML
```

```
    for page in root.findall('page'):
        page_id = page.find('id').text
        graph.add_node(page_id) # Add the page as a node

        # Find all outgoing links from the current page
        links_to = page.find('links_to')
        if links_to is not None:
            for link in links_to.findall('link'):
                target_id = link.text
                # Add a directed edge from the current page to the linked page
                graph.add_edge(page_id, target_id)

    return graph

def compute_topic_specific_pagerank(graph, topic_nodes, alpha=0.85):

    # Create the personalization vector.
    # It gives preference to the topic_nodes.
    personalization = {node: 0 for node in graph.nodes()}

    # Check if any topic nodes exist in the graph
    valid_topic_nodes = [node for node in topic_nodes if node in graph]
    if not valid_topic_nodes:
        print(f"Warning: None of the topic nodes {topic_nodes} were found in the graph. Using standard
PageRank.")
        return nx.pagerank(graph, alpha=alpha)

    # Assign equal preference to all valid topic nodes
    preference_value = 1.0 / len(valid_topic_nodes)
    for node in valid_topic_nodes:
        personalization[node] = preference_value

    print(f"\nComputing PageRank personalized for topic nodes: {valid_topic_nodes}")

    # Calculate PageRank with the personalization vector
    pagerank_scores = nx.pagerank(graph, alpha=alpha, personalization=personalization)

    return pagerank_scores

# --- Sample Data ---
# A sample XML structure representing a small collection of web pages.
# In a real scenario, this would be read from a .xml file.
sample_xml_data = """
<webgraph>
    <page>
        <id>sports_home</id>
        <links_to>
            <link>sports_news_1</link>
            <link>sports_news_2</link>
            <link>tech_home</link>
        </links_to>
    </page>
    <page>
        <id>sports_news_1</id>
        <links_to>
```

```
            <link>sports_home</link>
        </links_to>
    </page>
    <page>
        <id>sports_news_2</id>
        <links_to>
            <link>sports_home</link>
            <link>sports_news_1</link>
        </links_to>
    </page>
    <page>
        <id>tech_home</id>
        <links_to>
            <link>tech_news_1</link>
            <link>sports_home</link>
        </links_to>
    </page>
    <page>
        <id>tech_news_1</id>
        <links_to>
            <link>tech_home</link>
        </links_to>
    </page>
</webgraph>
"""

# --- Main Execution ---
if __name__ == "__main__":
    # 1. Parse the XML and generate the web graph
    web_graph = parse_xml_to_graph(sample_xml_data)

    print("--- Web Graph Generated ---")
    print(f"Nodes (Pages): {list(web_graph.nodes())}")
    print(f"Edges (Links): {list(web_graph.edges())}")

    # 2. Get user input for the topic
    print("\nPlease choose a topic to personalize PageRank.")
    topic_input = input("Enter a page ID (e.g., 'sports_home' or 'tech_home'): ")

    # For simplicity, we'll treat the single input as the topic.
    # This could be expanded to include multiple pages.
    topic_pages = [topic_input.strip()]

    # 3. Compute the topic-specific PageRank
    pagerank_results = compute_topic_specific_pagerank(web_graph, topic_pages)

    # 4. Display the results, sorted by rank
    print("\n--- Topic-Specific PageRank Results ---")
    sorted_pagerank = sorted(pagerank_results.items(), key=lambda item: item[1], reverse=True)

    for page, score in sorted_pagerank:
        print(f"{page:<15} | PageRank Score: {score:.4f}")
```

Output:

```
--- Web Graph Generated ---
Nodes (Pages): ['sports_home', 'sports_news_1', 'sports_news_2', 'tech_home', 'tech_news_1']
Edges (Links): [('sports_home', 'sports_news_1'), ('sports_home', 'sports_news_2'), ('sports_home',
'tech_home'), ('sports_news_1', 'sports_home'), ('sports_news_2', 'sports_home'), ('sports_news_2',
'sports_news_1'), ('tech_home', 'tech_news_1'), ('tech_home', 'sports_home'), ('tech_news_1', 'tech_home')]

Please choose a topic to personalize PageRank.
Enter a page ID (e.g., 'sports_home' or 'tech_home'):  tech_home

Computing PageRank personalized for topic nodes: ['tech_home']

--- Topic-Specific PageRank Results ---
tech_home       | PageRank Score: 0.3621
sports_home     | PageRank Score: 0.2869
tech_news_1     | PageRank Score: 0.1539
sports_news_1   | PageRank Score: 0.1158
sports_news_2   | PageRank Score: 0.0813
```

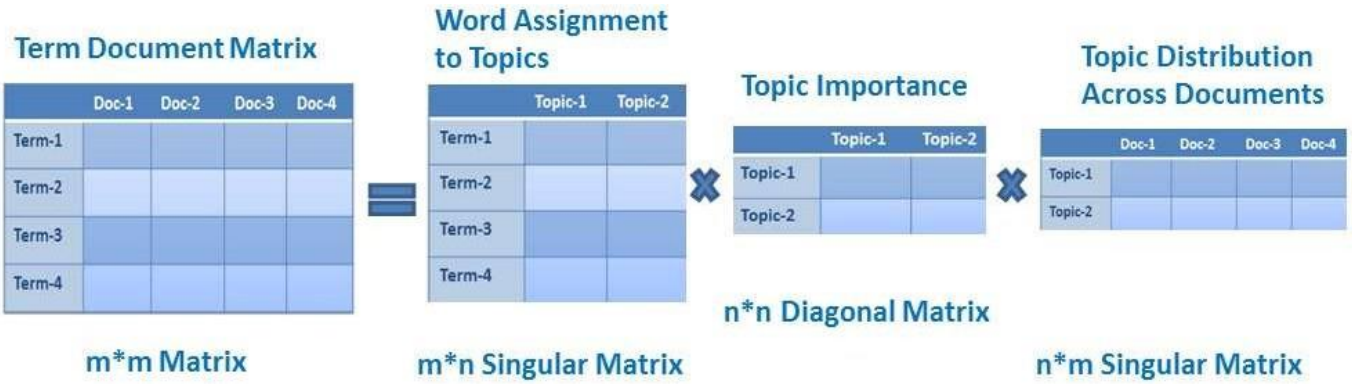
Experiment-8

8. Implement Matrix Decomposition and LSI for a standard dataset.

Aim: Matrix Decomposition and LSI for a standard dataset.

Procedure:

LSA (Latent Semantic Analysis) also known as LSI (Latent Semantic Index) LSA uses bag of word(BoW) model, which results in a term-document matrix(occurrence of terms in a document). Rows represent terms and columns represent documents. LSA learns latent topics by performing a matrix decomposition on the document-term matrix using Singular value decomposition. LSA is typically used as a dimension reduction or noise reducing technique.



Singular Value Decomposition(SVD)

SVD is a matrix factorization method that represents a matrix in the product of two matrices. It offers various useful applications in signal processing, psychology, sociology, climate, and atmospheric science, statistics and astronomy.

$$M=U\Sigma V^*$$

- M is an m×m matrix
- U is a m×n left singular matrix
- Σ is a n×n diagonal matrix with non-negative real numbers.
- V is a m×n right singular matrix
- V* is n×m matrix, which is the transpose of the V.

Identity matrix: It is a square matrix in which all the elements of the principal diagonal are ones, and all other elements are zeros.

Diagonal Matrix: It is a matrix in which the entries other than the main diagonal are all zero.

Singular Matrix: A matrix is singular if its determinant is 0 or a square matrix that does not have a matrix inverse.

Program:

```
import numpy as np
```

```
from sklearn.datasets import fetch_20newsgroups
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.decomposition import TruncatedSVD

# Load a subset of the 20 newsgroups dataset
categories = ['sci.space', 'comp.graphics', 'rec.sport.baseball']
newsgroups = fetch_20newsgroups(subset='train', categories=categories, remove=('headers', 'footers', 'quotes'))

# Convert the text data into TF-IDF matrix
vectorizer = TfidfVectorizer(stop_words='english', max_features=1000)
X_tfidf = vectorizer.fit_transform(newsgroups.data)
print("Original TF-IDF matrix shape:", X_tfidf.shape)

# Apply SVD for Latent Semantic Indexing (LSI)
n_components = 100  # Number of topics/dimensions to reduce to
svd = TruncatedSVD(n_components=n_components, random_state=42)
X_lsi = svd.fit_transform(X_tfidf)
print("Reduced matrix shape (after LSI):", X_lsi.shape)

# View the explained variance ratio
explained_variance = svd.explained_variance_ratio_.sum()
print(f"Total explained variance by {n_components} components: {explained_variance:.2f}")

# Example: Show topics (top terms per concept)
terms = vectorizer.get_feature_names_out()
for i, comp in enumerate(svd.components_[:5]):  # Show top 5 components
    terms_in_topic = [terms[j] for j in comp.argsort()[-10:]] # Top 10 terms
    print(f"Topic {i+1}: {' | '.join(terms_in_topic[::-1])}")
```

Output:

```
Original TF-IDF matrix shape: (1774, 1000)
Reduced matrix shape (after LSI): (1774, 100)
Total explained variance by 100 components: 0.37
Topic 1: space | just | like | don | think | know | year | good | time | does
Topic 2: year | team | game | games | pitching | think | hit | runs | win | baseball
Topic 3: space | nasa | launch | shuttle | moon | orbit | lunar | earth | station | sci
Topic 4: just | don | think | like | got | people | say | files | ve | problem
Topic 5: thanks | know | does | just | mail | space | advance | anybody | people | don
```

Experiment-9

9. Mining Twitter to identify tweets for a specific period (and/or from a geographical location) and identify trends and named entities.

Aim: Mining Twitter to identify tweets for a specific period (and/or from a geographical location) and identify trends and named entities.

Procedure:

Twitter Mining for Trends and Named Entities

Step 1: Set Up Twitter API Access

- **Register** for a Twitter Developer account at https://developer.twitter.com.
- **Create a Project** and **generate API Keys**:
  - API key
  - API key secret
  - Access token
  - Access token secret
- Use **Tweepy** (Python library) to access the Twitter API.

Step 2: Define Search Parameters

Set up the filters:

- **Keywords / Hashtags**
- **Date Range** (since and until)
- **Geolocation** (latitude, longitude, radius in km)
- **Language** (optional)

Step 3: Collect Tweets Using Tweepy

Step 4: Preprocess the Tweets

- Remove URLs, mentions, hashtags, emojis, and punctuations.
- Convert to lowercase.
- Tokenize and remove stopwords.

Step 5: Identify Trends (Frequent Terms/Hashtags)

Step 6: Named Entity Recognition (NER)

Step 7: Visualize Trends and Entities (Optional)

Program:

```
# This script performs text analysis on a public blog post about cloud computing
# to identify key trends and entities using the spaCy library.

import requests
from bs4 import BeautifulSoup
import spacy
from collections import Counter
```

```
# --- Setup and Data Scraping ---

def scrape_text_from_url(url):
    """
    Scrapes the main text content from a given URL.
    This function handles the network request and HTML parsing.

    Args:
        url (str): The URL of the web page to scrape.

    Returns:
        str: The extracted text content, or an empty string if an error occurs.
    """
    try:
        # Use a user-agent to mimic a real browser to avoid being blocked.
        headers = {'User-Agent': 'Mozilla/5.0'}
        response = requests.get(url, headers=headers)
        response.raise_for_status()  # Raise an exception for bad status codes (4xx or 5xx)

        # Parse the HTML content using BeautifulSoup
        soup = BeautifulSoup(response.text, 'html.parser')

        # Find all paragraph tags and join their text content
        paragraphs = soup.find_all('p')
        text = ' '.join([p.get_text() for p in paragraphs])

        return text

    except requests.exceptions.RequestException as e:
        print(f"Error fetching URL: {e}")
        return ""

# --- Natural Language Processing (NLP) ---

def analyze_text(text):
    """
    Analyzes the given text to find named entities and common words.

    Args:
        text (str): The text content to analyze.

    Returns:
        tuple: A tuple containing a list of named entities and a Counter
               object for common trends.
    """
    # Load the small English spaCy model.
    # If the model is not installed, run:
    # python -m spacy download en_core_web_sm
    try:
        nlp = spacy.load("en_core_web_sm")
    except OSError:
        print("SpaCy model 'en_core_web_sm' not found. Please run the following command to install it:")
        print("python -m spacy download en_core_web_sm")
        return [], Counter()
```

```
    doc = nlp(text)

    # Extract named entities (e.g., people, organizations, locations)
    # Filter out common, non-specific entities like "DATE" or "CARDINAL"
    named_entities = [(ent.text, ent.label_) for ent in doc.ents if ent.label_ not in ('DATE', 'CARDINAL',
'ORDINAL')]

    # Extract nouns and verbs to find common trends
    words = [token.lemma_.lower() for token in doc if token.pos_ in ['NOUN', 'VERB'] and not token.is_stop]
    trends = Counter(words)

    return named_entities, trends

# --- Main Execution ---

if __name__ == "__main__":
    # URL of a public blog post for text analysis.
    # This URL is a placeholder. You can replace it with any public URL
    # containing relevant text.
    url_to_scrape = 'https://x.com/googlecloud'

    print(f"Scraping text from: {url_to_scrape}\n")
    scraped_text = scrape_text_from_url(url_to_scrape)

    if scraped_text:
        # Analyze the scraped text
        entities, trends = analyze_text(scraped_text)

        # Display the results
        print("--- Named Entities ---")
        for entity, label in entities:
            print(f"- {entity} ({label})")

        print("\n--- Top 10 Trends (Nouns & Verbs) ---")
        for word, count in trends.most_common(10):
            print(f"- {word} ({count})")
    else:
      print("Scraping failed. No analysis performed.")
```

Output:

```
Scraping text from: https://x.com/googlecloud

--- Named Entities ---
- JavaScript (ORG)
- JavaScript (ORG)
- x.com (ORG)
- Help Center (GPE)
- X Corp. (ORG)

--- Top 10 Trends (Nouns & Verbs) ---
- browser (3)
```

- detect (1)
- enable (1)
- switch (1)
- continue (1)
- list (1)
- support (1)
- help (1)
- info (1)

Experiment-10

10. Implementation of PageRank on Scholarly Citation Network

Aim: Implementation of PageRank on Scholarly Citation Network

Procedure:

1. Data Collection

Collect a dataset representing scholarly citations. You can use:

- OpenCitations (https://opencitations.net/)
- Semantic Scholar Open Research Corpus
- Microsoft Academic Graph (MAG)
- DBLP Citation dataset

2. Build the Citation Graph

Represent the network as a directed graph, where:

- Nodes = scholarly papers
- Edges = citations (directed from *citing* to *cited* paper)

3. Preprocess the Graph

- Remove dangling nodes (nodes with no out-links), or handle them with damping.
- Normalize node identifiers.
- Optionally filter out low-cited or isolated papers.

4. Apply PageRank Algorithm

5. Interpret the Results

- Higher PageRank = more influential/important paper (i.e., cited by other important papers).
- Can be used to rank:
  - Authors
  - Papers
  - Journals
  - Institutions (if mapped accordingly)

6. Visualization

Program:

```
import networkx as nx
import matplotlib.pyplot as plt
# Create a directed graph
G = nx.DiGraph()
# Add citation relationships (A cites B: edge A → B)
edges = [
    ("Paper1", "Paper2"),
    ("Paper1", "Paper3"),
    ("Paper2", "Paper3"),
    ("Paper3", "Paper4"),
    ("Paper4", "Paper2"),
    ("Paper5", "Paper1"),
    ("Paper5", "Paper4"),
    ("Paper6", "Paper1"),
    ("Paper6", "Paper5"),
]

G.add_edges_from(edges)
# Compute PageRank
pagerank_scores = nx.pagerank(G, alpha=0.85)
```

```
# Display PageRank Scores
print("PageRank Scores for Scholarly Citation Network:")
for paper, score in pagerank_scores.items():
    print(f"{paper}: {score:.4f}")
# Optional: Visualize the graph with PageRank-based node sizes
plt.figure(figsize=(10, 7))
node_sizes = [v * 5000 for v in pagerank_scores.values()]
nx.draw_networkx(G, with_labels=True, node_size=node_sizes, node_color='lightblue',
arrowsize=20)
plt.title("Citation Network with PageRank Scores")
plt.show()
```

Output:

```
PageRank Scores for Scholarly Citation Network:
Paper1: 0.0508
Paper2: 0.2963
Paper3: 0.2985
Paper4: 0.2938
Paper5: 0.0356
Paper6: 0.0250
```

